

What the Fuck Is Going On Inside an LLM?

Выпуск 01. Железо для ИИ: от нагрузки до локального сервера

Атлас · аппаратная архитектура, обучение, инференс и данные · 26 сентября 2026

Статус. Это учебный выпуск к блоку 7 общего содержания. Формулы и числовые сценарии помогают сформулировать проверяемое требование к машине. Они не заменяют проверку конкретного checkpoint, версии драйвера и runtime. Цены и готовые корзины комплектующих намеренно не фиксируются: они требуют отдельного исследования рынка в день покупки. Исходный маршрут — 7.1–7.12.

Что читатель сможет сделать

1. Разложить задачу на подготовку данных, обучение, поиск, prefill и генерацию; понять, где нужны CPU, GPU, RAM, VRAM, SSD и сеть.
2. Приблизительно посчитать веса, KV-кэш и векторные данные, не смешивая гигабайты с гигабайтами.
3. Отличить требования инференса, полного обучения, LoRA/QLoRA и обслуживания одновременных запросов.
4. Составить паспорт нагрузки, проверить совместимость компонентов и получить измеримое «годится / не годится».

1. Машину выбирают под работу

Вопрос «какую модель запустит эта GPU?» неполон. Один и тот же checkpoint может загружаться в четырёхбитном формате для одиночного чата, не помещаться при длинном контексте и двадцати активных генерациях, а при полном обучении требовать совсем другой объём памяти. Модель также может *поместиться*, но работать медленнее допустимого из-за выгрузки слоёв в RAM.

Начните с паспорта: точная модель и ревизия, задача, формат весов, контекст, длина ответа, число активных последовательностей, целевые задержка и качество, размеры корпуса, операции записи и обновления, бюджет времени и денег. «20 пользователей» само по себе не равно двадцати одновременно удерживаемым KV-кэшам: это определяется поведением сервиса и планировщиком запросов.

Работа	Главный ресурс в типичном случае	Что измерять
OCR и разбор PDF	CPU, RAM, SSD; иногда GPU для OCR	Страниц/мин, ошибки извлечения, размер очереди
Эмбединги и reranking	GPU/CPU, RAM, пакетирование	Документов/с, качество поиска, пик памяти
Интерактивная генерация	VRAM, bandwidth, KV-кэш, CPU для обвязки	TTFT, inter-token latency, токенов/с
Пакетный инференс	GPU и планировщик, сеть/диск при подаче данных	Суммарный throughput при заданном качестве
Полное обучение	GPU, HBM/VRAM, межузловая сеть, хранилище	Токенов/с, шагов/с, восстановление после сбоя
LoRA/QLoRA	GPU-память под базу и активации, RAM/SSD	Пик памяти, время шага, качество адаптации

Модель ограничений. Для отдельного ядра Roofline связывает вычислительную мощность и пропускную способность памяти: при малой арифметической интенсивности ускорение вычислительных блоков почти не помогает. Для всей системы добавляются PCIe, сеть, декодирование корпуса и очереди. Это способ поставить гипотезу об узком месте, а не формула для точного предсказания токенов в секунду. FlashAttention показывает, что изменение перемещений между быстрой памятью чипа и HBM может улучшать attention без изменения его математической цели. См. [1–3].

Литература к разделу 1

Вводные работы

1. NVIDIA, *CUDA Programming Guide* (обновляемая документация). Иерархия памяти, исполнение на GPU и связь аппаратных возможностей с программной моделью. **Читать:** вводные разделы о programming model и memory; выписать, какие данные движутся между уровнями памяти. Официальный текст.
2. Samuel Williams, Andrew Waterman, David Patterson, *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures* (2009). Модель верхней границы производительности по compute и bandwidth. **Читать:** определение operational intensity и границы применимости графика. Первоисточник, Berkeley.

Специализированные работы

3. Tri Dao et al., *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness* (NeurIPS 2022). Пример проектирования алгоритма с учётом движения данных между SRAM и HBM. **Читать:** постановку IO-сложности и схему tiling; не переносить показатели ускорения на любую GPU. Публикация.

2. Память модели: веса, кэш и рабочие буферы

Первое приближение для плотной модели: сырые веса = $P \times b / 8$ байт, где P — число параметров, b — бит на вес. Для 8 млрд параметров и 4 бит это 4 млрд байт, или примерно 3,73 GiB. Фактический файл и использование памяти отличаются: scales, zero-points, таблицы квантования, незаквантизованные слои, runtime и временные буферы тоже занимают место. Количество *активных* параметров МоЕ за токен не равно общему объёму всех весов, которые нужно разместить или подгружать.

Для обычного полного KV-кэша decoder-only Transformer можно начать с

$$K = 2LH_{KV}DSBs,$$

где L — число слоёв, H_{KV} — число голов ключей/значений, D — размер головы, S — удерживаемые токены на последовательность, B — число одновременно удерживаемых последовательностей, s — байт на элемент. Множитель 2 означает key и value. При $L=32$, $H_{KV}=8$, $D=128$, $S=4096$, $B=1$, $s=2$ получаем 536 870 912 байт, то есть 0,5 GiB. Если таких последовательностей двадцать, сырая оценка — 10 GiB до накладных расходов менеджера кэша. GQA уже отражена в H_{KV} ; подстановка числа query heads зависит оценку.

У sliding-window attention, MLA и иных архитектур другая схема хранения. Prefix sharing, квантизация и offload меняют одновременно объём и задержку. Кэш является функцией конкретной реализации, а не только названия модели; сверяйте код и документацию backend. Hugging Face документирует разные стратегии кэша, а PagedAttention и vLLM показывают, почему управление фрагментацией и совместное использование блоков важны для сервиса [4–5].

Учебная таблица. Числа ниже — только нижняя граница сырых весов в 4 битах. Это не минимально необходимая VRAM и не обещание запуска.

Плотная модель	Сырые веса	Что ещё меняет результат
1B	0,5 GB	Формат, поддержка архитектуры, буферы
3B	1,5 GB	Контекст и квантование
8B	4 GB	KV-кэш и длина ответа
14B	7 GB	Версия backend и буферы
32B	16 GB	Offload, несколько GPU, стоимость обмена
70B	35 GB	Размещение всех весов и обмен между устройствами

Практическая граница. Установка `device_map=auto` или частичный GPU offload может помочь разместить модель, но перенос между RAM и VRAM меняет скорость. Не заключайте из успешной загрузки, что система удовлетворяет требованию к интерактивности [6].

Литература к разделу 2

Вводные работы

4. Hugging Face, *Cache strategies* (обновляемая документация). Dynamic, static, offloaded и quantized cache, а также ограничения отдельных реализаций. **Читать:** сравнительную таблицу стратегий и примеры; записать версию библиотеки. Документация.
5. Woosuk Kwon et al., *Efficient Memory Management for Large Language Model Serving with PagedAttention* (SOSP 2023). Управление KV-кэшем блоками в обслуживании запросов. **Читать:** источник фрагментации, отображение логических блоков и условия экспериментов. Работа.

Специализированные работы

6. Hugging Face, *Big Model Inference* (обновляемая документация). Размещение частей модели между ускорителем, CPU и при необходимости диском. **Читать:** `device_map`, `max_memory` и ограничения скорости offload. Документация.

3. Обучение требует другого бюджета

Во время полного обучения к весам добавляются градиенты, состояние оптимизатора и активации для backward. Схема с FP32 master weights, BF16 параметрами, FP32 градиентами и Adam-состояниями даёт иной расход, чем чистое BF16, 8-bit optimizer или шардирование. Поэтому «18 байт на параметр» — пример конкретной схемы, а не физическая константа. На практике размер активаций зависит от microbatch, длины последовательности, архитектуры, checkpointing и параллелизма.

При DDP каждый участник обычно держит копию модели; FSDP и ZeRO разделяют параметры, градиенты и/или состояния оптимизатора между устройствами. Это экономит память ценой коммуникаций и сложности восстановления. Tensor parallelism делит операции внутри слоя; pipeline parallelism распределяет последовательность слоёв; context/expert parallelism решает иные задачи. Сравнивать конфигурации надо по общей пропускной способности при одинаковом качестве и допустимой стоимости, а не по числу карт [7–8].

LoRA обучает малое число адаптерных параметров. QLoRA хранит базовую модель в квантованном виде и обучает адаптеры, но всё равно нуждается в памяти под активации, оптимизатор адаптера и временные тензоры. Существование эксперимента QLoRA на одной большой GPU не гарантирует, что любая модель и длина контекста поместятся на выбранной домашней карте [9–10].

Восстановление — часть архитектуры. Для длительного обучения важны скорость чтения данных, checkpoint, количество записей, достаточное место на SSD/object storage и тест восстановления после отключения. Максимальные FLOPS бессмысленны, если процесс регулярно простаивает на загрузке или теряет дни работы при сбое. В распределённой задаче паспорт должен содержать топологию сети и фактический объём обмена.

Литература к разделу 3

Вводные работы

7. PyTorch, *Getting Started with Fully Sharded Data Parallel (FSDP2)* (обновляемая документация). Разделение параметров, градиентов и состояния оптимизатора; применение к модели. **Читать:** пример `fully_shard`, порядок `sharding` и сохранение состояния. Документация.

Специализированные работы

8. Samyam Rajbhandari et al., *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models* (SC 2020). Исходная постановка устранения избыточных копий состояния обучения. **Читать:** уровни разделения и модель коммуникационных затрат. Материалы SC20.
9. Tim Dettmers et al., *QLoRA: Efficient Finetuning of Quantized LLMs* (NeurIPS 2023). Адаптация поверх квантизированной базы, NF4 и другие механизмы снижения памяти. **Читать:** схему памяти и ограничения экспериментов. Публикация.
10. Hugging Face, *Quantization with PEFT* (обновляемая документация). Практическая реализация адаптеров поверх квантованной модели. **Читать:** подготовку модели, целевые модули и ограничения совместимости. Документация.

4. Инференс: prefill, decode и очередь

Prefill обрабатывает входной контекст и создаёт начальные состояния; decode генерирует новые токены, многократно обращаясь к весам и кэшу. Для одного запроса эти фазы имеют разные профили работы. При пакетировании вес модели может обслуживать несколько последовательностей, но растут KV-кэш и ожидание в очереди. Управление блоками памяти, continuous batching, prefix caching и chunked prefill меняют компромисс между суммарной скоростью и задержкой отдельного пользователя [5, 11].

Измеряйте как минимум **TTFT** (до первого токена), **ITL/TPOT** (между токенами), end-to-end latency, суммарные output tokens/s, p95/p99 и долю ошибок/отказов. Для каждого опыта сохраните длину входа и ответа, частоту поступления запросов, политику batch, длину контекста, версию модели, драйвер и температуру устройства. Одинаковые средние tokens/s при разных хвостовых задержках могут означать различное качество сервиса.

Учебный случай. У нас 20 одновременно активных запросов с контекстом 4096 токенов. При параметрах раздела 2 сырой KV-кэш около 10 GiB. Если остальные 16 GiB заняты весами и буферами, карта с 24 GiB VRAM не обязана выдержать эту нагрузку. Возможны меньшая одновременность, иная архитектура, квантование кэша, больший ускоритель или очередь. Каждая мера меняет либо качество, либо задержку, либо цену; решение подтверждается тестом.

Литература к разделу 4

Вводные работы

11. vLLM, *Benchmark CLI* (обновляемая документация). Клиентское измерение TTFT, ITL и полной задержки. **Читать:** смысл каждой метрики, генерацию нагрузки и экспорт результатов. Документация.

Специализированные работы

12. vLLM, *Documentation* (обновляемая документация). Возможности планировщика, continuous batching, prefix caching и варианты параллелизма. **Читать:** архитектурные функции и условия поддерживаемого backend; затем проверить применимость к своей модели. Документация.

5. Компоненты локальной машины и совместимость

GPU. Смотрите сперва поддержку точной модели, формата и операций в нужном runtime; затем VRAM, пропускную способность памяти, численные форматы и устойчивую производительность. Версия CUDA и compute capability для NVIDIA, матрица ROCm с парой GPU/OC для AMD, Metal/MLX для Apple и XPU/SYCL либо иной backend для Intel определяют реальную применимость. Наличие общего GPU-бенчмарка не доказывает поддержку нужного kernel. Для AMD официальный список совместимости следует проверять по версии ROCm, а не по одному бренду карты [13–16].

CPU и RAM. CPU выполняет подготовку, OCR, токенизацию, индексирование, оркестрацию и часто часть offload. Важны потоки, пропускная способность RAM, PCIe lanes и стабильная скорость длительной работы. RAM должна вместить ОС, исходные данные, индексы, копии весов и буферы. Объём видеопамяти и объём RAM нельзя бездумно складывать как одну быструю память.

SSD, плата, питание. Рассчитайте место на модели, датасеты, индексы, checkpoints, временные файлы и резервные копии. Проверьте сокет, BIOS, физические слоты и их линии, длину и толщину GPU, разъёмы БП и теплоотвод. Два слота на плате не обещают две карты на полной скорости. Для сервера важны сеть, управление доступом, резервное копирование и мониторинг, а не только пиковая генерация.

Форма	Когда разумна	Основная проверка
Ноутбук/мини-ПК	Один пользователь, ограничение места и питания	Доступная память, устойчивые частоты, поддержка backend
Настольная станция	Локальные эксперименты, замена GPU/RAM, продолжительные задачи	Слоты, охлаждение, БП, шум, возможность расширения
Выделенный сервер	Постоянная многопользовательская нагрузка	Надёжность, очереди, сеть, наблюдаемость, восстановление
Аренда ускорителя	Редкое обучение или пиковый запуск	Полная стоимость, перенос данных, воспроизводимость окружения

Этот список задаёт выбор архитектуры, но не продаёт конкретный SKU. Для закупки к нему добавляют таблицу точных компонентов, официальный лист совместимости и пробный запуск выбранного backend.

Литература к разделу 5

Вводные работы

13. ggml-org, *llama.cpp* (обновляемый код и документация). CPU, CUDA, HIP, Metal, Vulkan и другие backend, включая частичный offload. **Читать:** supported backends, сборку и инструменты измерения; проверить конкретный commit. Репозиторий.
14. AMD, *ROCm Compatibility Matrix* (обновляемая документация). Соответствие поддерживаемых ускорителей, ОС и версии стека. **Читать:** выбранную версию матрицы перед покупкой AMD-карты. Документация.
15. NVIDIA, *Compute Capabilities* (обновляемая документация). Отображение моделей GPU на аппаратные возможности CUDA. **Читать:** строку конкретного GPU и требования выбранного runtime. Документация.

Специализированные работы

16. Apple ML Research, *MLX* (обновляемый код). Вычисления на Apple Silicon с единой моделью памяти. **Читать:** особенности unified memory и фактический расход задач; общий объём памяти не равен свободной памяти для модели. Репозиторий.

6. RAG: корпус, индекс и база данных тоже требуют железа

В RAG CPU и диск часто обрабатывают PDF, OCR и метаданные; GPU или CPU считают эмбединги; RAM и SSD держат поисковый индекс, текст и журнал обновлений; отдельный GPU может выполнять reranking и генерацию. Ускорение LLM само по себе не устранил медленную индексацию или плохую разметку документов.

Для N векторов размерности d из s байт на компонент сырые значения занимают $N \times d \times s$. Учебный пример: 150 000 фрагментов $\times$ 1024 измерения $\times$ 4 байта = 614 400 000 байт, или 0,572 GiB. В реальности добавятся структура HNSW/IVFFlat, первичный текст, метаданные, индексы фильтров, журнал, копии и временная память на построение. Из «3000 PDF» нельзя вывести N : сначала нужна выборка по страницам, OCR и правило разбиения.

pgvector предоставляет точный поиск по умолчанию и приближённые индексы HNSW/IVFFlat; выбор влияет на скорость, recall, время сборки и память [17]. Для академического или корпоративного корпуса проверьте не только среднюю задержку top-k, но и качество ссылок на точные страницы, фильтры доступа, обновление версии документа и восстановление после сбоя.

Три независимых бюджета: хранилище исходников и резервных копий; RAM/SSD индекса и метаданных; VRAM и вычисления моделей. Их можно размещать на одном компьютере или разделять по машинам. Решение зависит от числа одновременных запросов и от того, какие данные можно передавать наружу.

Литература к разделу 6

Вводные работы

17. pgvector, *README: Indexing and Exact/Approximate Search* (обновляемая документация). Типы индексов, выборка и параметры памяти/поиска. **Читать:** exact search, HNSW, IVFFlat и примеры измерения размеров индекса. Репозиторий.

7. Паспорт нагрузки и решение о покупке

Ниже шаблон, который надо заполнить до выбора конфигурации. Его машиночитаемая версия и учебный калькулятор приложены к выпуску. Калькулятор считает только нижние оценки трёх массивов и не выводит «совместимо».

Поле	Пример записи
Задача	RAG по 3000+ PDF/Excel, один исследователь или 15 членов команды
Модели	Точные ID и ревизии генератора, embedding, OCR и reranker
Нагрузка	Распределение входных/выходных токенов, частота и активные генерации
Качество	Набор вопросов, точность ссылок, допустимая потеря после квантизации
Сервис	TTFT, ITL, p95, допустимая очередь и время восстановления
Данные	Страницы, фрагменты, размерность, история версий, резервные копии
Стек	ОС, драйвер, runtime, формат весов, настройки batch и KV
Цена	Покупка, электричество, хранение, поддержка, аренда на пике

Порядок выбора. (1) Получить baseline на имеющемся оборудовании. (2) Оценить память с запасом и проверить поддержку backend. (3) Назвать 2–3 допустимых класса машины, включая аренду, если обучение редко. (4) Протестировать точную модель и корпус на целевых длинах и конкурентности. (5) Выбирать по качеству, p95 и полной цене; протокол сохранить рядом со спецификацией. Для ноутбука с 6 GiB VRAM разумный первый опыт — маленькая модель и короткий контекст, а не заранее заявленная поддержка 8B при любой квантизации.

Контроль отказа. Если запуск падает, различите отсутствие поддержки операции, OOM весов, рост KV, нехватку RAM, ограничения файловой системы и неправильную сборку. Если работает медленно, разделите prefill/decode, проверьте CPU offload, использование памяти и частоты под длительной нагрузкой. Изменяйте один фактор за опыт.

8. Практикум и исследовательский фронтир

Опыт А — память. Возьмите 1–3B и 7–8B модели с опубликованными параметрами архитектуры. Предскажите размеры весов и KV для 1, 4 и 20 последовательностей; запишите остаток памяти после загрузки и пик при генерации. Объясните расхождение с формулой.

Опыт В — скорость. На одном checkpoint сравните CPU, GPU и частичный offload при фиксированных входах 512/4096 токенов и выходах 128/512. Сохраните TTFT, ITL, p95, энергопотребление и версии стека. Не сравнивайте throughput разных моделей как доказательство превосходства железа.

Опыт С — RAG. Измерьте реальные страницы и фрагменты выборки, посчитайте индекс, время OCR, embedding и поиска. Сравните точный поиск с HNSW при одинаковом наборе вопросов и целевом recall; проверьте страницы цитат.

Опыт D — обучение. На небольшой модели сравните полный шаг, LoRA и QLoRA при одинаковых данных и оценке качества. Зафиксируйте peak VRAM, шагов/с, размер checkpoints и возможность восстановления.

Фронтирные темы для продолжения: разделение prefill и decode между устройствами; disaggregated serving; точность и цена квантизации KV; динамическая маршрутизация MoE; совместное размещение поиска и генерации; отказоустойчивое многопользовательское обслуживание; алгоритмы с меньшим движением данных; согласованное измерение

энергии на задачу. Каждое улучшение проверяется при одинаковом качестве и полной стоимости, а не только по одному бенчмарку.

Для телеметрии GPU можно использовать официальные счётчики DCGM, учитывая поддержку поля конкретным устройством [18]. Результат выпуска — паспорт нагрузки, файл расчёта, совместимая спецификация, журнал пробного запуска и обоснованное решение по классу машины. Следующий выпуск сможет сравнить конкретные компоненты и цены на дату закупки.

Литература к разделу 8

Специализированные работы

18. NVIDIA, *DCGM Exporter Metrics* (обновляемая документация). Метрики использования памяти, температуры, мощности и ограничений GPU. **Читать:** семантику полей и поддержку профилирования на выбранном устройстве. Документация.

Границы выпуска. Проверены указанные официальные страницы, репозитории и страницы работ; глубина чтения отдельных академических текстов варьирует от аннотации до разделов, о чём сказано в журнале выпуска. Числа в примерах вычислены из формул и не являются замерами на реальной машине. Конкретная корзина комплектующих, совместимость каждого checkpoint и рыночные цены требуют отдельного протокола испытаний.